

FLAG: An FPGA-Based System for Low-Latency GNN Inference Service Using Vector Quantization

Yunki Han, Taehwan Kim, Jiwan Kim, Seohye Ha, Lee-Sup Kim

School of Electrical Engineering, KAIST

Daejeon, Republic of Korea

{yunki.han, th.kim, jiwan_kim, seohye, leesup}@kaist.ac.kr

Abstract—Enabling real-time GNN inference services requires low end-to-end latency to meet service level agreements. However, intensive preparation steps and the neighborhood explosion problem pose significant challenges to efficient GNN inference serving. In this paper, we propose FLAG, an FPGA-based GNN inference serving system using vector quantization. To reduce preparation overhead, we introduce offline preprocessing to precompute and compress hidden embeddings for serving. A dedicated FPGA accelerator leverages the precomputed data to enable lightweight aggregation. As a result, FLAG achieves average speedups of 154 $\times$, 176 $\times$, and 333 $\times$ on three GNN models compared to the baseline system.

Index Terms—Graph Neural Network, Hardware Accelerator, Large-scale Graph

I. INTRODUCTION

The performance of applications based on graph-structured data has been significantly improved through the adoption of Graph Neural Networks (GNNs) [1]–[4]. Since GNNs have been effectively applied to graphs across various domains, including social media, e-commerce, and financial services, numerous real-world applications can now be implemented using GNNs. Consequently, graph databases integrated with GNN frameworks can support web services such as recommendation systems [5], [6], traffic prediction [7], and fraud detection [8], [9].

Despite the promising potential of GNN-powered applications, previous works on GNN accelerators and systems have paid less attention to the practical implementation of real-time services. Real-time GNN inference systems must achieve sufficiently low latency to meet Service Level Agreements (SLAs) for each arrival of inference request. However, prior efforts to accelerate GNN inference have commonly focused on full-batch inference [10]–[13], which has a significant gap from the requirements of practical real-time services. Industrial graphs often contain billions of nodes, with sizes reaching terabyte-scale or beyond. In such cases, computing the entire graph to obtain inference results for requested nodes is highly inefficient.

To address this inefficiency, computing the inference results only for the requested target node batch is a practical approach, widely adopted across both academia and industry. The workflow of this GNN inference serving system can be divided into two steps: the preparation step and the computation step. Industrial-scale graphs are typically vast and dynamic, often managed by graph databases. During the preparation step, the system retrieves the computational graph and input node embeddings from the graph database. Subsequently, in the computation step, these input data for the GNN models are transferred to accelerator devices (e.g., GPUs) and processed to produce inference results.

Although this solution appears reasonable, there are two significant challenges that hinder the achievement of low-latency serving. First, the entire graph is often much larger than the available device memory, causing the preparation step to suffer from low host memory bandwidth and limited I/O bus bandwidth. As a result, the preparation step consumes a dominant portion of the end-to-end latency. The second challenge is the neighbor explosion problem [2], [14], [15]. In

each GNN layer, nodes aggregate information from their neighbors, and stacking multiple GNN layers leads to an exponential increase in the number of required input nodes. This issue significantly raises computation and memory access costs, resulting in prolonged latency.

To address this problem, we first identify the specific characteristics of real-time, user-facing GNN services. Real-time graphs are dynamic, with nodes and edges being continuously added or deleted. However, from the perspective of each timestamp when a request arrives, the portion of target nodes occupies only an extremely small part of the entire graph. The remaining part of the graph becomes static after new nodes are added. For this reason, we focus on these existing nodes, dubbed background nodes, for efficient inference.

In this paper, we propose FLAG, an FPGA-based low-latency GNN inference serving system that employs vector quantization. To minimize heavy data transfer, hidden embeddings of background nodes are precomputed and compressed using vector quantization during the offline preprocessing phase. For online GNN inference serving, we introduce a lightweight aggregation method that leverages the precomputed background nodes to eliminate the neighbor explosion problem. The dedicated FLAG hardware is designed and implemented on an FPGA to support both computation and embedding updates. As a result, FLAG system achieves average speedups of 154 $\times$, 176 $\times$, and 333 $\times$ on three GNN models compared to the baseline system.

II. BACKGROUND AND MOTIVATION

A. Fundamentals of GNNs

A Graph Neural Network (GNN) is a category of neural network models designed to handle graph-structured inputs. A common characteristic of modern GNN models is the graph convolution operation, which conducts aggregation of features from adjacent nodes. In each GNN layer, node embeddings are updated through two distinct computation phases: *aggregation* and *combination*.

$$a_v^l = \text{Aggr}(\{h_u^{l-1} : u \in N(v)\}), \quad h_v^l = \text{Comb}(h_v^{l-1}, a_v^l) \quad (1)$$

As shown in Equation 1, firstly, the target node v gathers embeddings (h_u^{l-1}) from its neighbor nodes ($N(v)$) and reduce them into single embedding (a_v^l) using an aggregation function (Aggr). The aggregation function, which is predefined according to the specific GNN model, can be operations such as sum, mean, or weighted sum. The output of the aggregation phase is then transformed into a new embedding (h_v^l) in a different dimensional space using a combination function (Comb), typically implemented as matrix multiplication.

$$X^l = \sigma(C^l X^{l-1} W^l) \quad (2)$$

Most of GNN models can be simply formulated as matrix multiplication, as shown in Equation 2. $X^{(l)}$ represents the node embedding matrix resulting from the l -th layer. In each GNN layer, node embeddings are aggregated and transformed through multiplication by a convolution matrix C^l and a weight matrix W^l . For instance, if we apply GCN [1] to this model, the convolution matrix becomes the normalized adjacency matrix $C = \tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2}$.

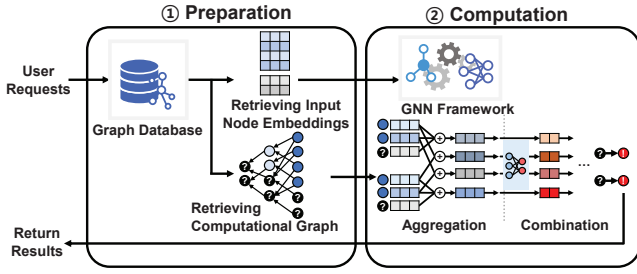


Fig. 1. Real-time GNN inference system workflow.

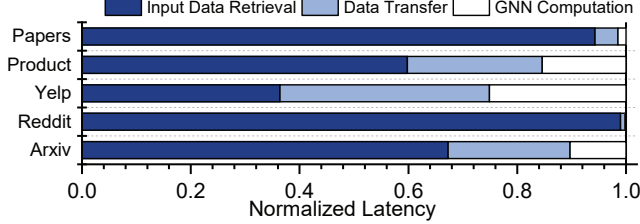


Fig. 2. Breakdown of GNN inference

B. Real-time GNN Inference Workflow on Large-scale Graph

Graph data in industrial domains is typically extremely large and dynamic, so graph-structured data is commonly managed using graph databases. Real-time GNN inference serving solutions [16] are implemented using GNN frameworks integrated with these graph databases. Although GNNs have a wide range of applications, this paper specifically focuses on node classification.

In the given solution, we can divide the end-to-end inference workflow into two distinct steps. Figure 1 illustrates the abstraction of the real-time GNN inference system workflow. When a user request arrives for target nodes, the system prepares the necessary input data, including input node embeddings and the computational graph, from the graph database for GNN model computation. These data are then packed and sent to the GNN framework for inference, and the inference results are subsequently returned to the user. The detailed process of serving inference results is described as follows. Important processes are highlighted using numbering.

The overall workflow begins with a user transaction or behavior that typically affects the existing graph, prompting the system to **① update the graph**. During this step, newly added nodes and edges may be inserted into or deleted from the graph data. Following these updates, **② subgraphs are extracted** to construct the computational graph for the target nodes, preparing the input for the GNN model. Additionally, **③ node embeddings are retrieved** from the graph database according to the computational graph. These two sets of input data are then **④ transferred to the accelerator devices**. Finally, the devices, supported by the GNN framework, **⑤ compute the GNN model** and return the inference results.

Figure 2 shows the portion of each step in the end-to-end GNN inference workflow. The Input Data Retrieval process encompasses steps 1 to 3, while Data Transfer and GNN Computation correspond to steps 4 and 5, respectively. The experiment was conducted using a 2-layer GraphSAGE model implemented with the DGL framework. In most cases, the preparation phase—comprising input data retrieval and data transfer—dominates the end-to-end latency. Notably, input data retrieval accounts for the largest portion across all datasets, consuming up to 99% of the overall latency. This analysis highlights the critical need to mitigate the substantial preparation time to achieve low-latency, practical GNN inference serving. Therefore, our work proposes a direction to alleviate this challenge.

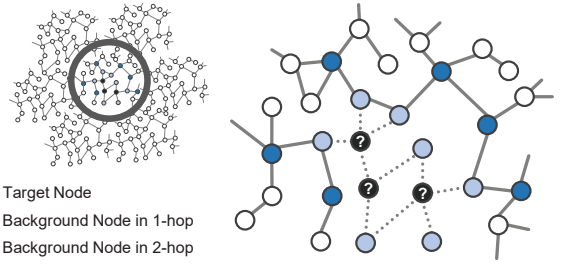


Fig. 3. Target nodes and background nodes

C. Opportunities in Background Nodes

The challenges of real-time GNN services arise from the need to handle dynamic, large-scale graphs efficiently. When the graph is static and includes the target nodes—referred to as the transductive setup—inference can be performed during the training phase. In this case, the system only needs to retrieve the inference output from the database for each request without additional computation. However, in real-time, user-facing environments, requests often involve newly added nodes. This inductive nature of GNN inference serving introduces the aforementioned challenges.

Although the graph is dynamic, at each time step, only the portion containing the target nodes and their edges significantly differs from the previous time step. In other words, most of the graph remains relatively static over small time intervals. In this paper, we refer to these existing nodes as **background nodes**, borrowing and extending the concept from [17]. As illustrated in Figure 3, target nodes occupy only a small portion of the entire graph and require 2-hop neighboring background nodes for computing inference results. This implies opportunities to accelerate GNN inference. If unseen, newly added nodes are requested, most of the input operand nodes are already known, existing nodes. Therefore, we leverage offline preprocessing of these background nodes through precomputation and compression methods before the service.

D. Efficient GNN Inference Serving with Data Compression

Caching compressed data in device memory is an intuitive solution to reduce data transfer, but careful selection of the compression method and targets is essential to achieve this objective. For instance, some previous works [17], [18] employ graph condensation for efficient GNN inference serving. This approach condenses the original graph into a smaller synthetic graph that maintains similar representational properties. Nodes that are not target nodes are compressed using condensation, thereby reducing memory requirements and lowering computation costs. However, this approach is unsuitable for real-time, user-facing environments. Because graph condensation loses the detailed structure of the original graph, it cannot handle dynamic graphs that require frequent updates.

To address this problem and achieve our goal, we employ vector quantization, a widely used data compression method, to reduce dimensionality in our GNN inference serving system. In contrast to graph condensation, which compresses both structural and feature data, vector quantization focuses solely on the node embedding matrix. Given a set of vectors, the process begins with finding centroid vectors, known as codewords, through k-means clustering. Once the codebook, which consists of these codewords, is prepared, the original vectors are replaced by the index of the closest codeword. Previous works [19], [20] have applied vector quantization to address similar challenges, but primarily in the context of training. In this paper, we propose a vector quantization-based solution specifically designed for a practical inference environment.

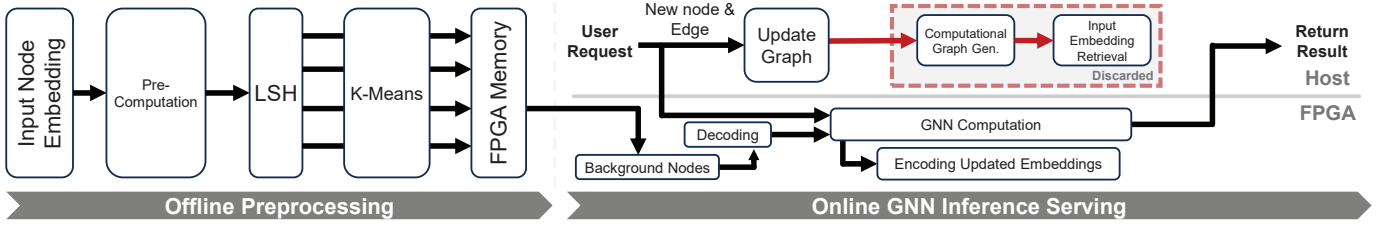


Fig. 4. Overview of the FLAG System Workflow

III. PROPOSED WORK

In this section, we introduce our proposed system, FLAG, an FPGA-based solution for low-latency GNN inference service utilizing vector quantization.

A. System Overview

As shown in Fig. 4, the overall workflow of the FLAG system is divided into offline preprocessing and online GNN inference serving. Before the service begins, hidden embeddings of existing nodes, which act as background nodes during inference, are precomputed and compressed. The compression method employs vector quantization, incorporating locality-sensitive hashing to consider data locality and layout. Once preprocessing is complete, online GNN inference serving is performed on the FLAG system. A dedicated accelerator architecture is deployed on the FPGA, efficiently handling both computation and embedding updates. Furthermore, to mitigate the neighbor explosion problem, we propose a lightweight aggregation method that leverages precomputed hidden embeddings. As a result, the proposed FLAG system achieves low-latency GNN inference service by addressing the aforementioned challenges.

B. Vector Quantization with Locality-Sensitive Hashing

The objective of offline preprocessing is to precompute the inference results for background nodes and compress them so that they fit into the device memory. Figure 5 illustrates the FLAG offline preprocessing process, and Algorithm 1 provides a detailed description.

Given the input embeddings of background nodes X_B^0 and the pretrained GNN model, we first precompute the hidden embeddings of the background nodes (lines 1-5). During this step, we store the hidden embeddings after multiplying them by the weight matrix (line 3). Since the output dimension is generally smaller than the input dimension, this reduces the storage overhead for the precomputed outputs. The precomputation for each layer includes an additional step to generate threshold vectors via a column-wise mean operation on the hidden embeddings (line 4), which are subsequently used for hashing nodes into sub-groups.

After completing the precomputation stage, vector quantization is applied. Our vector quantization method is a modified form of product quantization. Initially, each embedding matrix is divided into M submatrices to minimize computation costs and reconstruction errors (line 6). The row vectors in each submatrix are then compared with the previously acquired thresholds, and the subvectors are hashed into $2^{(f/M)}$ buckets (line 9). This process is similar to locality-sensitive hashing using random projection, providing two key benefits for performance optimization. First, the online encoding process for updated or newly added embeddings can be significantly lightweight due to the hash function. Second, data locality is improved by allocating similar codewords to adjacent physical addresses. Given that GNN datasets exhibit high correlation [17], [21], [22] among

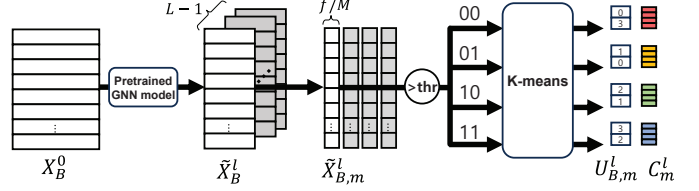


Fig. 5. FLAG offline preprocessing

Algorithm 1 FLAG Offline Preprocessing Algorithm

INPUT: Input Embeddings X_B^0 , Weight Matrices $W^1, \dots, W^L$
OUTPUT: Index Matrices $U_{B,m}^l$, Codebooks C_m^l

```

1: for  $l$  in 1 to  $L$  do
2:    $X_B^l \leftarrow \text{GNNLayer}^l(X_B^{(l-1)})$ 
3:    $\tilde{X}_B^l \leftarrow X_B^{(l-1)} W^l$ 
4:    $\text{thr}^l \leftarrow \tilde{X}_B^l.\text{column\_wise\_mean}()$ 
5: end for
6:  $\tilde{X}_{B,m}^l, \text{thr}_m^l := \tilde{X}_B^l[\frac{(m-1)f^l}{M} : \frac{mf^l}{M}], \text{thr}^l[\frac{(m-1)f^l}{M} : \frac{mf^l}{M}]$ 
7: for  $l$  in 1 to  $L-1$  do
8:   for  $m$  in 1 to  $M$  do
9:      $\text{idx}_m^l \leftarrow \text{element\_wise\_compare}(\tilde{X}_{B,m}^l, \text{thr}_m^l)$ 
10:    for  $i$  in 0 to  $2^{(f/M)} - 1$  do
11:       $u_i \leftarrow \text{idx}_m^l == i$ 
12:       $U_{B,m}^l[u_i], C_m^l[iM : (i+1)M] \leftarrow k\_means(\tilde{X}_{B,m}^l[u_i])$ 
13:    end for
14:  end for
15: end for
16: return  $U_{B,m}^l, C_m^l$ 

```

connected nodes, nodes with similar embeddings are likely to be clustered together. After hashing, the subvectors within the same buckets are encoded using k-means clustering (line 12). Consequently, the codebooks C_m^l and the index matrices $U_{B,m}^l$ are returned.

With this offline preprocessing step, we obtain a compressed representation of the original large-scale embedding matrices. This addresses one of the challenges associated with heavy data retrieval on the host side and the substantial data transfer to the device. However, the compressed data includes not only the input node embeddings but also the precomputed hidden embeddings. The role of these precomputed hidden embeddings is introduced in the next subsection.

C. Light-weight Aggregation with Precomputed Background Nodes

To achieve low-latency inference, one of the biggest challenges, the neighbor explosion problem, must also be addressed. In this subsection, we propose a lightweight aggregation method that leverages precomputed background hidden embeddings and efficient approximated computations.

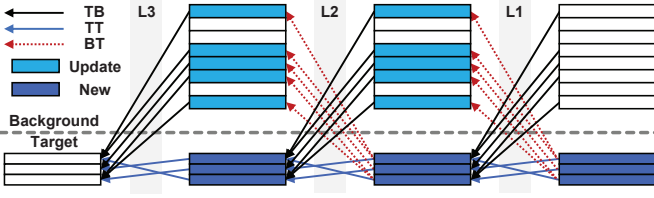


Fig. 6. Target node aggregation and Background node update

The baseline system generates the computational graph for the target nodes before inference and retrieves the input node embeddings for the first layer of the GNN model. In this process, the receptive field of the target nodes expands to their L -hop neighbors. The memory access and operation complexity of the first layer is $O(bd^L)$, where b denotes the batch size of the target nodes, d denotes the average degree, and L denotes the number of layers. In the FLAG system, the hidden embeddings of the background nodes are precomputed, eliminating the need for most of the aggregation computation that leads to the neighbor explosion problem. Although it may seem that all the issues causing the neighbor explosion problem have been resolved, one latent issue remains.

The problem is that while background nodes contribute to the hidden embeddings of target nodes, the target nodes also influence the background nodes. For clear explanation, let us denote a target node as $\mathbf{T}$ and a background node as $\mathbf{B}$. Figure 6 illustrates an example computational graph of a 3-layered GNN model. To avoid confusion, we illustrate the embedding matrices of the background nodes in their decoded form.

For the target nodes, the aggregation is straightforward, involving the aggregation of input node embeddings from the target nodes and their neighboring nodes in the background. Equation 3 describes this operation, which we refer to as *target aggregation*. Two types of aggregation, including TB (black line) and TT (blue line), are also illustrated in Figure 6.

$$X_T^l \leftarrow \text{Aggr}^l(X_T^{l-1}, \tilde{X}_{B \cap N(T)}^{l-1}) \quad (3)$$

Before a target node is added to the existing graph, the hidden embeddings of the background nodes are complete. However, once the target node is added, not only do the background nodes influence the target node, but the target node also affects the background nodes, as illustrated in Figure 6. Consequently, the current values of the background nodes connected to the newly added target node must be updated. Equation 4 describes this operation, referred to as the *background update*. In Figure 6, BT (red line) represents this type of aggregation.

$$\tilde{X}_{B \cap N(T)}^l \leftarrow \text{Aggr}^l(\tilde{X}_{B \cap N(T)}^l, X_T^{l-1}) \quad (4)$$

The proposed lightweight aggregation method, consisting of target aggregation and background update, maximizes the benefits of pre-computation, enabling the avoidance of unnecessary aggregation. In addition, the most significant advantage of this method is that it does not require an L -hop propagated computational graph. Consequently, we can eliminate the computational graph generation process and the exponential increase in both memory access and computational complexity.

Despite the advantages of the proposed aggregation method, it does have two drawbacks. The first issue is that some background updates, which need to be scattered from the updated background nodes, may be neglected when the layer depth is 3 or more. As more GNN layers are stacked, errors can accumulate. However, no noticeable

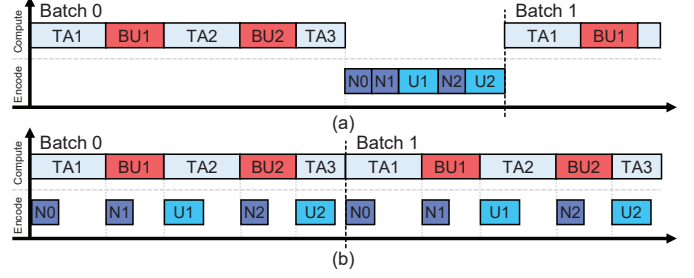


Fig. 7. Computation and encoding (a) without pipelining, and (b) with pipelining

impact on accuracy has been observed when the number of layers is 3. Since GNN models typically do not use a layer depth of 4 or more, this issue is not a significant concern. The second drawback is the need for encoding updated and newly introduced embeddings, which introduces overhead due to the use of a vector-quantized compressed format. To mitigate potential performance degradation, we design hardware that supports both computation and the encoding/decoding processes without introducing additional latency.

D. FLAG Hardware Architecture Design

Computation and Encoding Pipeline. The objective of the FLAG architecture design is to support the proposed computation framework while minimizing the overhead associated with non-computation processes, such as embedding encoding for updates. To achieve this, the FLAG architecture is divided into two main blocks: the computation block and the encoding block. As described in the previous subsection, the proposed computation flow necessitates encoding the embeddings of updated background nodes and newly added target nodes. This encoding process would not be a concern if inference were performed only once. However, we assume a scenario where continuous inference requests are received.

Figure 7 illustrates two scenarios: (a) without pipelining and (b) with pipelining. In this example, TA denotes Target Aggregation, and BU denotes Background Update, while N and U represent New and Update, respectively. For clarity, the color scheme used is similar to that in Figure 6. As shown in Figure 7, the latency of the encoding step accumulates on top of the computation latency when pipelining is not used. If the next request batch arrives before batch 0 has completed processing, stalls occur. However, with pipelining, the encoding latency is hidden within the computation latency. From the perspective of latency, this effectively eliminates the overhead of encoding.

FLAG Architecture Overview. Figure 8 describes an overview of the FLAG architecture. To enable computation and encoding pipelining, the architecture is divided into a computation engine and an encoding engine. The computation engine consists of two main units: the combination unit and the aggregation unit. The combination unit is implemented using a systolic array to efficiently combine the embeddings of the requested target nodes. The aggregation unit is a SIMD array specialized for vector computation, executing both target aggregation and background updates. When precomputed hidden embeddings are used, the decoder reads the codeword corresponding to each node's code index and concatenates them to form the input operands for the aggregation unit. Several types of buffers store different data sets, such as the input buffer, weight buffer, and edge buffer. The global buffer is primarily used for storing unencoded hidden embeddings but can also store various types of data for multiple purposes.

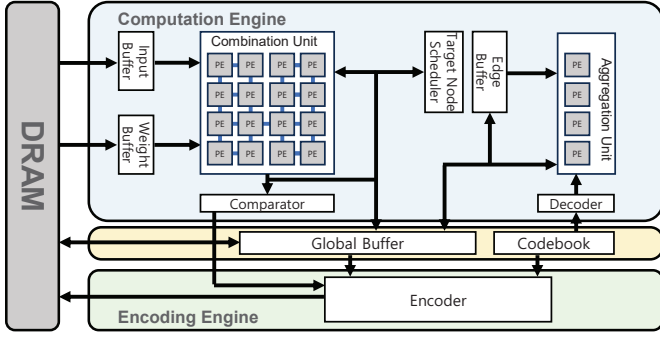


Fig. 8. FLAG Hardware Architecture

Target node scheduling and encoding. When the embeddings of a target node batch arrive at FLAG, the first step is the combination phase, which involves projecting the embeddings through multiplication with the weight matrix. Because we utilize a threshold-based compression method during offline preprocessing, the binary code produced by thresholding the outputs from the combination unit in the comparator becomes highly useful. First, this binary code can be employed by the encoder to search for the most similar codeword. Since we use the same thresholds and the same projection matrix (the weight matrix) during online serving, FLAG enables lightweight encoding without requiring complex comparisons. Another way to leverage this binary code is to schedule the computation order of the target nodes within a batch. The characteristics of the GNN dataset are also applicable to newly added target nodes. In other words, target nodes hashed into the same buckets are likely to exhibit similar connection patterns. By reordering these nodes, FLAG enhances the temporal locality of computing nodes. Since symmetric TB aggregation and BT update are performed at each layer, the impact of target node scheduling is effectively doubled.

IV. EVALUATION

A. System Configuration for Experiments

Baseline System. The baseline system consists of a machine with 2 Intel Xeon Silver 4210R CPUs (384GB) and 4 NVIDIA RTX 3090 GPUs (24GB, 936GB/s). We implement the baseline workflow similar to [16] using the Deep Graph Library (DGL) [23].

FPGA system setup. The FLAG system consists of a machine with 2 Intel Xeon Gold 6240 CPUs (384GB) and AMD Xilinx u55c FPGA card (16GB, 460GB/s) [24]. The FLAG hardware design is implemented on a U55C card using Vitis and Vivado. For comparison, we also evaluate the state-of-the-art real-time inference accelerator, FlowGNN [25]. The experimental system environment for FlowGNN is identical to that used for FLAG.

B. Benchmarks

We select node property prediction benchmark datasets Arxiv, Products, Papers [26], Reddit [2], and Yelp [27] containing more than 100K nodes. Table I shows detailed information about these benchmark datasets. The GNN models used for evaluation include three representative architectures: GCN [1], GraphSAGE [2], and GAT [3]. In this work, the hidden embedding dimension for all models is set to 128, and the size of heads for GAT is set to 16. To avoid the stochastic fluctuation of the inference accuracy, we do not use neighbor sampling during evaluation. The default batch size is 128, and the model depth is 2.

TABLE I
TESTBENCH DATASETS

Dataset	Nodes	Edges	Input dim.	Classes
Arxiv	169,343	1,166,243	128	40
Reddit	232,965	114,848,857	602	41
Yelp	716,847	13,954,819	300	100
Products	2,449,029	61,859,140	100	47
Papers	111,059,956	1,615,685,872	128	172

TABLE II
ACCURACY AND COMPRESSION RATIO

Model	Arxiv		Reddit		Yelp		Products		Papers	
	CR	Acc(%)	CR	Acc(%)	CR	Acc(%)	CR	Acc(%)	CR	Acc(%)
GCN	Baseline	1	71.5	1	95.6	62.8	1	77.6	1	65.8
	FLAG	37.1	71.3	173.7	95.5	45.2	62.5	18.5	77.6	16.8
GraphSAGE	Baseline	1	71.1	1	95.9	63.6	1	78.3	1	66.2
	FLAG	37.1	71.1	195.8	96.1	45.2	63.4	18.5	78.2	16.8
GAT	Baseline	1	71.3	1	95.3	63.1	1	79.5	1	63.5
	FLAG	20.7	71.0	91.8	95.1	36.1	62.7	9.3	79.3	8.4

C. Accuracy and Compression Ratio

For the proposed approach to be practical, sufficient compression must be achieved without significant accuracy degradation. To ensure this, we determine the marginal compression ratio that results in less than a 0.5% accuracy loss. The compression ratio is calculated by dividing the size of the original embedding matrix by the size of the vector-quantized data. Table IV-C presents the data compression results, obtained using 2-layer GNN models, except for Products and Papers, which use 3-layer models.

The Reddit dataset achieves the highest compression ratio. This is because the FLAG compression method compresses the dimension-reduced embeddings, obtained by multiplying the original embeddings with the weight matrix, rather than compressing the original embeddings directly. As a result, Reddit's large input dimension of 602 significantly contributes to its compression ratio. Conversely, the Papers and Products datasets experience a disadvantage in compression ratio due to the increase in input dimensions for similar reasons.

D. End-to-end System Latency

We evaluate the end-to-end latency of the FLAG system in comparison to the baseline system and the FlowGNN system. Since the FLAG system employs offline preprocessing to avoid heavy preparation time, it significantly outperforms the existing systems. As shown in Figure 9, the FLAG system achieves up to 3224 $\times$, 3245 $\times$, 3322 $\times$ speedup for each GNN models compared to the baseline system, with an average speedup of 154 $\times$, 176 $\times$, and 333 $\times$. The Reddit dataset, which has the largest input dimension (602) and the highest average degree (493), exhibits particularly remarkable speedup. In contrast, the system equipped with the state-of-the-art accelerator, FlowGNN, achieves only a modest speedup of up to 1.25 $\times$. This result underscores the importance of comprehensive, systemic solutions to address the aforementioned challenges for achieving low-latency GNN inference service.

E. Computation only latency

To prove FLAG is also optimized to low latency GNN computation, we evaluate the GNN computation time compared to baseline and FlowGNN. Since the FLAG architecture replace the most portion of aggregation by precomputation, it significantly outperforms not only GPU but also FlowGNN. As shown in Figure 10, the FLAG architecture achieves up to 381 $\times$, 318 $\times$, 1281 $\times$ speedup for each GNN models compared to the baseline system, with an average speedup of 186 $\times$, 151 $\times$, and 468 $\times$. FlowGNN demonstrates significant speedup compared to the baseline GPU for computing GNN inference on mini-batches, but it fails to achieve similar results on the Reddit dataset. FlowGNN is specialized for low-latency processing of

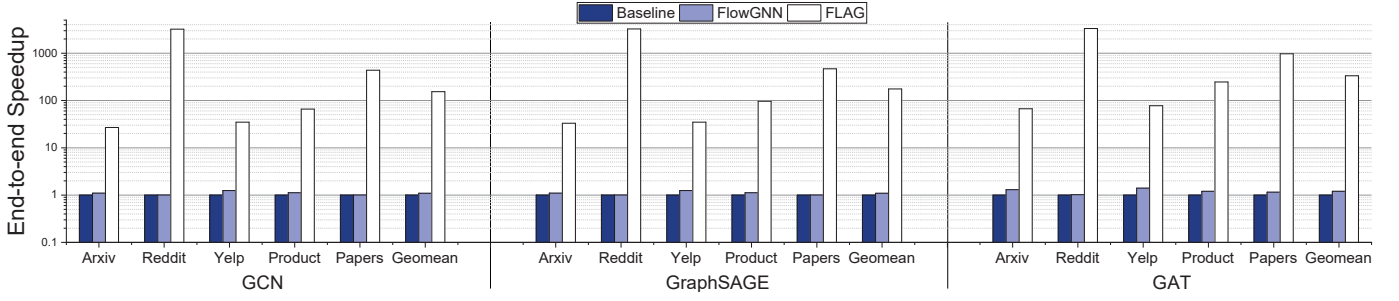


Fig. 9. End-to-end speedup of three system

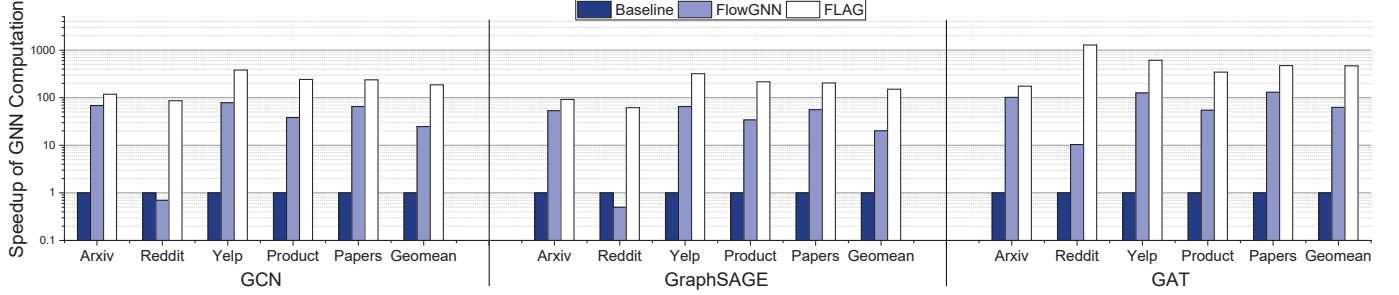


Fig. 10. Speedup of GNN computation

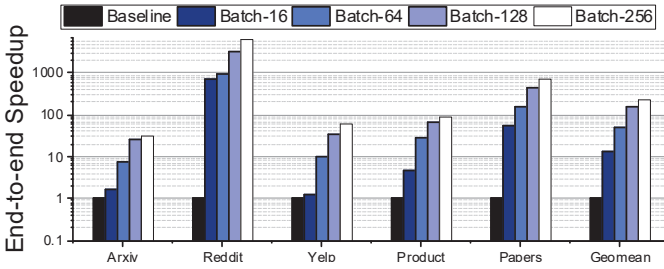


Fig. 11. End-to-end speedup according to batch size

streams of relatively small graphs. However, under our experimental condition, which samples full neighbors, nearly full-batch inference is required for the first layer of the Reddit dataset. In contrast, FLAG achieves superior performance on all GNN models compared to FlowGNN, as its precomputation-based lightweight aggregation method effectively avoids the neighborhood explosion problem.

F. Effect of Batch Size

We perform the evaluation using different batch sizes, ranging from 16 to 256, on the GraphSAGE model. All latency measurements are normalized to the baseline, which is evaluated under the same conditions as in Figure 9. As shown in Figure 11, the end-to-end speedup increases as the batch size becomes larger. Since the complexity of the neighbor explosion problem is $O(bd^l)$, a larger batch size results in greater preparation overhead. Consequently, the latency portion of the preparation step increases as the batch size grows, eventually constituting the majority of the end-to-end latency. The speedup of FLAG is achieved by minimizing the heavy preparation step.

To address this problem, neighbor sampling can be used to reduce the average degree d to a sample size s . However, this sampling method probabilistically loses important information and does not fundamentally solve the neighborhood explosion problem, as it remains exponentially proportional to s . On the other hand, our precomputation-based lightweight aggregation efficiently addresses this issue.

V. RELATED WORK

Constructing an efficient machine learning application serving system is a crucial problem in the AI era. GNN applications are also applied to critical domains where time inefficiency can lead to severe consequences, such as fraud detection [8], [9] in financial services. This work focuses on a single FPGA-equipped server node; however, many cases of computing environments, such as fog computing [28] or cloud computing [29], should be considered to transition toward real-world systems. To this end, both hardware optimization and algorithm optimization are needed. Although they are not addressed in this paper, many methods can be applied orthogonally to other ideas. Examples include quantization techniques [30], [31] and efficient edge pruning methods [32], [33]. [34] and [35] offer effective solutions for co-optimization, but questions remain regarding their adoption for real-time services.

VI. CONCLUSION

The practical implementation of GNN inference services on large-scale industrial graphs is a challenging problem due to the need for low end-to-end latency to meet service level agreements. In this paper, to enable low-latency GNN inference serving, we analyzed the end-to-end workflow and identified the bottlenecks of the baseline system. Most of the latency is consumed by the inevitable preparation step, and the neighborhood explosion problem causes latency to increase exponentially with the number of layers. To address the two primary challenges, we proposed FLAG, an FPGA-based system for low-latency GNN inference service using vector quantization. In the offline preprocessing step, we introduced a hidden embedding pre-computation and vector quantization method combined with locality-sensitive hashing. For the online GNN serving step, we designed a dedicated hardware architecture for FPGA, which supports our proposed ideas. As a result, we achieved average speedups of $154\times$, $176\times$, and $333\times$ on three GNN models compared to the baseline.

ACKNOWLEDGMENT

This research was supported by Samsung Advanced Institute of Technology (SAIT), Samsung Electronics Co., Ltd.

REFERENCES

- [1] T. N. Kipf and M. Welling, "Semi-Supervised Classification with Graph Convolutional Networks," in *Proceedings of the 5th International Conference on Learning Representations*, 2017.
- [2] W. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Advances in Neural Information Processing Systems* (I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, eds.), vol. 30, Curran Associates, Inc., 2017.
- [3] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," *6th International Conference on Learning Representations*, 2017.
- [4] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?," in *7th International Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6-9, 2019*, OpenReview.net, 2019.
- [5] X. He, K. Deng, X. Wang, Y. Li, Y. Zhang, and M. Wang, "Lightgcn: Simplifying and powering graph convolution network for recommendation," in *Proceedings of the 43rd International ACM SIGIR conference on research and development in Information Retrieval*, pp. 639–648, 2020.
- [6] R. Ying, R. He, K. Chen, P. Eksombatchai, W. L. Hamilton, and J. Leskovec, "Graph convolutional neural networks for web-scale recommender systems," in *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*, pp. 974–983, 2018.
- [7] S. Guo, Y. Lin, N. Feng, C. Song, and H. Wan, "Attention based spatial-temporal graph convolutional networks for traffic flow forecasting," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 33, pp. 922–929, 2019.
- [8] Z. Liu, C. Chen, X. Yang, J. Zhou, X. Li, and L. Song, "Heterogeneous graph neural networks for malicious account detection," in *Proceedings of the 27th ACM international conference on information and knowledge management*, pp. 2077–2085, 2018.
- [9] D. Wang, J. Lin, P. Cui, Q. Jia, Z. Wang, Y. Fang, Q. Yu, J. Zhou, S. Yang, and Y. Qi, "A semi-supervised graph attentive network for financial fraud detection," in *2019 IEEE international conference on data mining (ICDM)*, pp. 598–607, IEEE, 2019.
- [10] M. Yan, L. Deng, X. Hu, L. Liang, Y. Feng, X. Ye, Z. Zhang, D. Fan, and Y. Xie, "Hygcn: A gcn accelerator with hybrid architecture," in *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, pp. 15–29, IEEE, 2020.
- [11] T. Geng, A. Li, R. Shi, C. Wu, T. Wang, Y. Li, P. Haghi, A. Tumeo, S. Che, S. Reinhardt, and M. C. Herbordt, "Awb-gcn: A graph convolutional network accelerator with runtime workload rebalancing," in *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pp. 922–936, 2020.
- [12] J. Li, A. Louri, A. Karanth, and R. Bunescu, "Genax: A flexible and energy-efficient accelerator for graph convolutional neural networks," in *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pp. 775–788, 2021.
- [13] R. Hwang, M. Kang, J. Lee, D. Kam, Y. Lee, and M. Rhu, "Grow: A row-stationary sparse-dense gemm accelerator for memory-efficient graph convolutional neural networks," in *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pp. 42–55, 2023.
- [14] W.-L. Chiang, X. Liu, S. Si, Y. Li, S. Bengio, and C.-J. Hsieh, "Cluster-gcn: An efficient algorithm for training deep and large graph convolutional networks," in *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*, pp. 257–266, 2019.
- [15] J. Chen, J. Zhu, and L. Song, "Stochastic training of graph convolutional networks with variance reduction," in *International Conference on Machine Learning*, pp. 942–950, PMLR, 2018.
- [16] H. W. Jian Zhang and M. Zhu, "Build a gnn-based real-time fraud detection solution using amazon sagemaker, amazon neptune, and the deep graph library," <https://aws.amazon.com/cn/blogs/machine-learning/build-a-gnn-based-real-time-fraud-detection-solution-using-amazon-sagemaker-amazon-neptune-and-the-deep-graph-library/>, 2022. Accessed: (2024-11-06).
- [17] L. Cao, H. Deng, Y. Yang, C. Wang, and L. Chen, "Graph-skeleton: 1% nodes are sufficient to represent billion-scale graph," in *Proceedings of the ACM Web Conference 2024, WWW '24*, (New York, NY, USA), p. 570–581, Association for Computing Machinery, 2024.
- [18] S. Si, F. Yu, A. S. Rawat, C.-J. Hsieh, and S. Kumar, "Serving graph compression for graph neural networks," in *The Eleventh International Conference on Learning Representations*, 2023.
- [19] M. Ding, K. Kong, J. Li, C. Zhu, J. Dickerson, F. Huang, and T. Goldstein, "Vq-gnn: a universal framework to scale-up graph neural networks using vector quantization," in *Proceedings of the 35th International Conference on Neural Information Processing Systems, NIPS '21*, (Red Hook, NY, USA), Curran Associates Inc., 2024.
- [20] Y. Ma, P. Gong, T. Wu, J. Yi, C. Yang, C. Li, Q. Peng, G. Xie, Y. Bao, H. Liu, and Y. Xu, "Eliminating data processing bottlenecks in gnn training over large graphs via two-level feature compression," *Proc. VLDB Endow.*, vol. 17, p. 2854–2866, Aug. 2024.
- [21] D. Lim, F. Hohne, X. Li, S. L. Huang, V. Gupta, O. Bhalerao, and S. N. Lim, "Large scale learning on non-homophilous graphs: New benchmarks and strong simple methods," *Advances in Neural Information Processing Systems*, vol. 34, pp. 20887–20902, 2021.
- [22] J. Zhu, Y. Yan, L. Zhao, M. Heimann, L. Akoglu, and D. Koutra, "Beyond homophily in graph neural networks: Current limitations and effective designs," *Advances in neural information processing systems*, vol. 33, pp. 7793–7804, 2020.
- [23] M. Wang, D. Zheng, Z. Ye, Q. Gan, M. Li, X. Song, J. Zhou, C. Ma, L. Yu, Y. Gai, T. Xiao, T. He, G. Karypis, J. Li, and Z. Zhang, "Deep graph library: A graph-centric, highly-performant package for graph neural networks," 2020.
- [24] X. Inc., "Alveo u55c high performance compute card," 2024. Accessed on May 01, 2024.
- [25] R. Sarkar, S. Abi-Karam, Y. He, L. Sathidevi, and C. Hao, "Flowgcn: A dataflow architecture for real-time workload-agnostic graph neural network inference," in *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pp. 1099–1112, IEEE, 2023.
- [26] W. Hu, M. Fey, M. Zitnik, Y. Dong, H. Ren, B. Liu, M. Catasta, and J. Leskovec, "Open graph benchmark: Datasets for machine learning on graphs," *arXiv preprint arXiv:2005.00687*, 2020.
- [27] H. Zeng, H. Zhou, A. Srivastava, R. Kannan, and V. K. Prasanna, "Graphsaint: Graph sampling based inductive learning method," in *8th International Conference on Learning Representations, ICLR 2020, Addis Ababa, Ethiopia, April 26-30, 2020*, OpenReview.net, 2020.
- [28] L. Zeng, P. Huang, K. Luo, X. Zhang, Z. Zhou, and X. Chen, "Fograph: Enabling real-time deep graph inference with fog computing," in *Proceedings of the ACM Web Conference 2022, WWW '22*, (New York, NY, USA), p. 1774–1784, Association for Computing Machinery, 2022.
- [29] H. Hu, F. Liu, Q. Pei, Y. Yuan, Z. Xu, and L. Wang, "grapher: A resource-efficient serverless system for gnn serving through graph sharing," in *The Web Conference 2024*, 2024.
- [30] Z. Liu, K. Zhou, F. Yang, L. Li, R. Chen, and X. Hu, "EXACT: Scalable graph neural networks training via extreme activation compression," in *International Conference on Learning Representations*, 2022.
- [31] S. A. Tailor, J. Fernandez-Marques, and N. D. Lane, "Degree-quant: Quantization-aware training for graph neural networks," *arXiv preprint arXiv:2008.05000*, 2020.
- [32] Z. Liu, K. Zhou, Z. Jiang, L. Li, R. Chen, S.-H. Choi, and X. Hu, "Dspar: An embarrassingly simple strategy for efficient gnn training and inference via degree-based sparsification," *Transactions on Machine Learning Research*, 2023.
- [33] H. Seo, J. Yun, and E. Yang, "Teddy: Trimming edges with degree-based discrimination strategy," in *The Twelfth International Conference on Learning Representations*.
- [34] Z. Zhu, F. Li, G. Li, Z. Liu, Z. Mo, Q. Hu, X. Liang, and J. Cheng, "Mega: A memory-efficient gnn accelerator exploiting degree-aware mixed-precision quantization," in *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pp. 124–138, IEEE, 2024.
- [35] C. Chen, K. Li, X. Zou, and Y. Li, "Dygnn: Algorithm and architecture support of dynamic pruning for graph neural networks," in *2021 58th ACM/IEEE Design Automation Conference (DAC)*, pp. 1201–1206, 2021.